



Task 4 - Threading, Polishing and Interfacing

In the previous tasks, you implemented the complete processing pipeline step by step: from ArUco detection and coordinate transformations to trajectory comparison and offset analysis. However, a functional prototype alone is not sufficient for a robust real-time system. The final implementation must also be structured, synchronized, stable, and user-friendly.

Task 4 therefore focuses on integrating all previously developed components into a complete application architecture. This task is divided into two parts:

- **Task 4.1 — Threading and Implementing ArUco Positions** Focuses on restructuring the system using multithreading, queues, and synchronization events to ensure stable real-time processing.
- **Task 4.2 — Polishing and Interfacing** Focuses on improving the overall software quality, usability, readability, visualization, and interface behavior of the final implementation.



Task 4.1 - Threading and Implementing ArUco Positions

In Task 3, you created the functions to describe the `ideal_approach` trajectory and compared it to the aircraft's actual position. For this, you used three arguments: `global_coordinates`, `appAngle`, and `tdwnDistance`. Up to this point, these were arbitrary placeholders, independent of the implementation from Tasks 0–2.

One might assume that integrating this code would be straightforward. However, this is not the case. The main challenge lies in the fundamentally different operating speeds of the two processes: (a) the camera feed and (b) the calculation of offsets. The detection pipeline runs continuously in real time, while the offset computation operates on discrete data samples. If both processes are executed sequentially, the system may experience frame drops, delayed calculations, or a buildup of unprocessed data.

To resolve this issue, you will restructure your implementation using **thread-based execution with synchronization mechanisms**. Threading allows multiple parts of a program to run concurrently, enabling the detection process to operate independently from the calculation process without blocking execution.

To simplify this restructuring, the functions in Tasks 0–3 were already clearly defined within their respective scopes. Each component was implemented as an independent, modular function with well-defined inputs and outputs. This separation ensures that detection, transformation, and offset calculation logic remain isolated, logical and reusable.

In Conclusion: In this task, you will replace the placeholders you created in Task 3. To achieve this, you will explore and implement threading. This will require significant restructuring of your code.

Thread Synchronization Concept

Instead of directly sharing variables between the camera processing thread and the analysis thread, you will use **synchronization events and thread-safe communication structures**.

The correct approach is:

- Each thread runs independently
- Data is transferred using a **queue-based buffer**
- Execution control is handled using **event flags**

→ **IMPORTANT:** While doing so global variables are not allowed! Why?

A shared global variable approach may seem simple, but it introduces critical problems:

- Data can be overwritten before it is processed
- Threads may access partially updated values
- Execution order becomes unpredictable

This leads to unreliable offset calculations and unstable system behavior, especially in real-time applications such as camera-based detection.



Correct Thread Communication Model

Your implementation must follow this structure:

1. The **camera thread** captures frames and performs detection.
2. The results are stored in a **thread-safe queue**.
3. The **analysis thread** retrieves data from the queue and performs offset calculations.
4. A **synchronization event** controls when processing is allowed or paused.

Required Implementation Changes

To correctly implement threading in this task, you must modify your system as follows:

- Replace all shared global state variables with a **Queue-based data pipeline**
- Introduce at least one **threading.Event()** for execution control
- Ensure that the camera and analysis processes run in separate threads
- Prevent both threads from modifying the same data structure simultaneously

Expected Thread Architecture

For this Task your solution should include:

- `cam_event_loop()` — captures frames and performs detection
- `analyse_pos_loop()` — calculates offsets from queued data
- `main thread` — handles user input and plotting
- `correct structure` — a code structure following the guideline described in "Correct Thread Communication Model"

Communication between these components must be handled exclusively via:

- **Queues** for data transfer
- **Events** for execution control



Task 4.2 - Polishing and Interfacing

After completing the threaded implementation, the final step is to transform your project into a deployment-ready system. The focus of this task is no longer the mathematical functionality itself, but the quality, structure, and presentation of your final software solution.

This task evaluates how professionally and efficiently your implementation is prepared for execution on the Raspberry Pi platform.

Main Goals

- Polished code structure implemented in a dedicated deployment branch
- Polished and minimalistic console interface

1. RaspberryDeployment Branch

Create a new branch named:

RaspberryDeployment

This branch must contain a **cleaned, optimized, and deployment-ready** version of your project.

This branch should contain *only* the files and code required for execution on the Raspberry Pi.

The goal is to create a lightweight and efficient deployment structure.

This means:

- unnecessary files must be removed
- outdated scripts and experiments must be deleted
- unused functions and dead code must not remain
- old folders, temporary files, screenshots, recordings, and unused calibration assets must not be present
- processing that can be performed externally on a computer should not be executed on the Raspberry Pi

You are expected to make thoughtful engineering decisions regarding what is truly required for deployment.

Unnecessary files, redundant data, or poor repository organization will result in score reduction.

Rewriting and restructuring parts of your code is expected and will likely be necessary to achieve a clean deployment architecture.



2. Polished Console Interface

The final implementation should present itself as a clean and professional real-time system.

As such your console output should feel like a finalized interface rather than a debugging environment.

Design Goal:

Video Input → Offset Output

The system should behave like a black box:

- video data enters the system
- only the relevant offset information is displayed

The console output should therefore contain only:

- lateral offset
- vertical offset

Any unnecessary console spam, debug information, excessive logging, or irrelevant outputs will result in score reduction.

Interface Expectations

You are encouraged to improve the visual quality and readability of the interface.

Possible improvements include:

- colored terminal output
- aligned and structured live-updating values
- stable formatting against rolling data
- clean refresh behavior
- status indicators
- visually appealing formatting
- additional creative improvements

Creativity and presentation quality will be evaluated as part of this task.

The objective is to create a minimal, clean, and professional operator interface focused entirely on the essential system outputs.



Final Note

This task requires significant restructuring, cleanup, and engineering effort. **A quick or improvised solution will not be sufficient.**

Approach it with a realistic development mindset: plan carefully, simplify where possible, and continuously refine your implementation until it is stable, efficient, and deployment-ready. Testing and iteration are expected parts of the process.

You will be given adequate time to complete this properly. **Use it.**

Happy Coding!